

Dawuro Platform — Backend Specification

Version 2.0 · 27 August 2026 · supersedes API_CONTRACT v1 (GhanaMediaWatch)

0. How to use this document

This is the complete specification for the Dawuro backend. **Both clients are already built** — an Expo mobile app for reporters and a Next.js console for institutions and platform operators. They run today against fixtures. This document describes the server they need.

It is written to be handed to an AI coding assistant in full. It is deliberately self-contained: every type, every constant and every rule appears here literally, so nothing has to be inferred from a repository the assistant cannot see.

Normative language. **MUST** is a requirement — breaking it is a defect. **SHOULD** is a strong recommendation with room for a documented alternative. **MAY** is genuinely optional.

Read §4 and §14 before writing any code. §4 is the trust model, which is the product; §14 is the list of things that must never happen. Everything else is mechanics.

Language. All shapes are given as TypeScript because both clients are TypeScript and these exact declarations are compiled against today. Implement the server in whatever you like; the JSON on the wire must match these shapes exactly.

What already exists

LAYER	STATE
Mobile app (Expo / React Native)	Built. Capture, GPS gating, offline outbox, resumable upload, feed, comments, earnings. 214 tests.
Web console (Next.js 15)	Built. Business inbox, routing desk, editorial workbench, onboarding, approvals, payouts, organisations.
Shared business rules (<code>@dawuro/core</code>)	Built and unit-tested. ~3,600 lines. Reproduced in full in §3–§11 below.
Backend	Nothing. This document.

The one rule about shared logic

Routing, commission, billing, permissions, assurance classification and the editorial gate are implemented in the clients and specified here. **The server MUST implement them independently and treat its own answer as authoritative.** Client-side rules exist to make the interface honest, not to secure anything — anyone can call your endpoints directly. Where this document gives an algorithm, implement that algorithm; do not trust a client to have applied it.

1. What Dawuro is

A Ghanaian incident-reporting platform. Members of the public film incidents on a phone. Reports are automatically routed to institutions — government agencies, utilities, media houses — whose declared interests and geography match. Those institutions license reports, which pays the reporter a commission, and may then publish a report to the public feed credited to themselves.

`dawuro` is the gong an Akan town crier strikes to gather people before speaking.

The loop

```
reporter films → provenance sealed at capture → auto-routed to matching institutions
      ↑                                     ↓
commission paid ← institution licenses ← editorial verifies → published to feed
```

Four roles

ROLE	SURFACE	WHAT THEY DO
<code>reporter</code>	Mobile only	Films incidents, submits, earns commission. Never uses the console.
<code>business</code>	Console only	Receives routed reports, licenses them, assigns to staff, publishes.
<code>platform_owner</code>	Console only	Operates the service: onboards institutions, oversees routing, runs payouts.
<code>editor</code>	Console only	Decides whether a claim is true. Deliberately separate from <code>platform_owner</code> — operating the service and adjudicating truth are different jobs.

A reporter who signs into the console is shown an explanation and no data.

Why the trust model is the product

Anyone can collect footage. The reason an institution would pay for Dawuro rather than accept WhatsApp videos is that Dawuro can say *where a file came from* and *how far anyone has got in checking what it shows* — and can prove it never conflated the two. \$4 is that model. If the backend gets one thing exactly right, it is \$4.

2. Conventions

2.1 Base URL and versioning

```
https://api.dawuro.gh/v1
```

The version is in the path. Additive changes (new optional field, new enum member accompanied by a client release) do not bump it. Anything that would break a shipped client does.

Clients in the field cannot be assumed to update. Every enumeration in this document is a **closed union** in client code, so a server returning an unknown enum value is a client crash, not a graceful degradation. **Do not add enum values without a coordinated client release.**

2.2 Authentication

Three kinds of credential.

Device token. Issued by `POST /devices` with no prior auth. Identifies an installation, not a person. Sufficient to read the public feed and to submit anonymously. This is what makes anonymous reporting real rather than cosmetic.

User token. Issued by the auth endpoints. Identifies a person. Required for earnings, personal history, and anything an account owns.

Organisation context. A user token carries zero or more organisation memberships. Requests acting on behalf of an institution **MUST** carry `X-Dawuro-Org: {businessId}`, and the server **MUST** verify the caller actually holds a membership in that organisation with the required capability (§8.4). Never infer the organisation from anything else.

```
Authorization: Bearer {token}
X-Dawuro-Org: biz_01JBX7Q2K9      # only when acting as an institution
```

Access tokens **SHOULD** be short-lived (~1 hour) with refresh tokens rotated on use. The console stores its session in an **httpOnly cookie** so browser JavaScript can never read it; when the backend lands, the backend access token is held server-side inside that session and attached by Next.js route handlers. **The browser must never see the backend token.**

2.3 Required headers

HEADER	ON	PURPOSE
<code>Authorization</code>	all but <code>POST /devices</code>	<code>Bearer {token}</code>
<code>X-Dawuro-Org</code>	institution endpoints	Which organisation is acting
<code>Idempotency-Key</code>	all POST that create	UUIDv4. See §2.6
<code>X-Client-Version</code>	all	e.g. <code>mobile/1.0.0</code> , <code>console/1.0.0</code>
<code>X-Request-Id</code>	all	UUIDv4 from the client; echo it back in every response and error

2.4 Content types

`application/json; charset=utf-8` everywhere, except chunk upload bodies which are `application/octet-stream`.

2.5 Errors

Every non-2xx response **MUST** use this envelope. No bare strings, no HTML error pages, no variation.

```
{
  "error": {
    "code": "GPS_ACCURACY_REJECTED",
    "message": "Location accuracy of 210m exceeds the 100m limit for this category.",
    "requestId": "req_01JBX7Q2K9",
    "details": { "accuracyM": 210, "limitM": 100 }
  }
}
```

`message` is shown to users. Write it in plain English, name the actual problem, and say what to do. `details` is optional and free-form.

```
export type ApiErrorCode =
  | 'VALIDATION_FAILED'           // 400
  | 'TOKEN_EXPIRED'               // 401  client refreshes and retries
  | 'TOKEN_INVALID'               // 401  client signs out
  | 'FORBIDDEN'                   // 403
  | 'INCIDENT_NOT_FOUND'          // 404
  | 'IDEMPOTENCY_CONFLICT'        // 409
  | 'UPLOAD_EXPIRED'              // 410  client restarts from POST /incidents
  | 'MEDIA_TOO_LARGE'             // 413
  | 'GPS_ACCURACY_REJECTED'        // 422
  | 'MEDIA_HASH_MISMATCH'         // 422  client re-uploads
  | 'RATE_LIMITED'                // 429
  | 'INTERNAL'                    // 500
  | 'MAINTENANCE'                 // 503
  | 'NETWORK_UNAVAILABLE';        // client-side only, never sent by the server
```

Retryability drives the mobile outbox and MUST be correct. These four are retryable: `RATE_LIMITED`, `INTERNAL`, `MAINTENANCE`, `NETWORK_UNAVAILABLE`. Everything else is permanent. Returning a permanent code for a transient failure strands a report on someone's phone forever; returning a retryable code for a permanent one makes thousands of devices retry hourly for eternity.

2.6 Idempotency

Every creating POST MUST accept `Idempotency-Key` and MUST be safe to replay. The mobile outbox retries aggressively over bad connections, and a duplicated report means a duplicated commission payment.

Store the key with the response for **at least 24 hours**. A replay with the same key returns the original response and status. A replay with the same key but a *different* body returns `409 IDEMPOTENCY_CONFLICT`.

2.7 Pagination

Cursor-based everywhere. Never offsets — reports change state constantly, and an offset page 2 would skip and duplicate rows as items clear review.

```
export interface Page<T> {
  items: T[];
  nextCursor: string | null;
  hasMore: boolean;
}
```

Cursors are opaque to the client. Default page size 20, maximum 100.

2.8 Rate limiting

Return `429 RATE_LIMITED` with a `Retry-After` header in seconds. The client honours it. Suggested starting points: 60 writes/minute per device, 600 reads/minute per token, 5 `POST /devices` /hour per IP.

2.9 Time

All timestamps are **ISO 8601 UTC with milliseconds**: `2026-08-27T14:31:00.000Z`. Never local time, never a bare date, never a Unix integer. Fields ending `Iso` are strings in this format.

The server clock is authoritative. §5.4 covers what to do when a device clock disagrees.

2.10 Coordinates

WGS84 decimal degrees. Latitude `-90..90`, longitude `-180..180`, six decimal places (~11 cm). Distances in **metres**, always integers, always named `...M` or `...Metres`.

2.11 Money — read this twice

Every monetary amount in this system is an integer number of pesewas. 100 pesewas = 1 Ghana cedi (GHS). Fields are named `...Pesewas` without exception.

- **Never** use a float, a decimal string, or a `Number` holding cedis anywhere in the system — not in the database, not in JSON, not in an intermediate calculation.
- **Never** round in the client. The server computes; the client displays.
- Where a calculation involves multipliers (commission, \$10.2), **round once at the end**. Rounding at each step compounds error across a payout batch, and a ledger that does not balance is worse than no ledger.
- Reconciliation is by subtraction, not by a second rounding:
`reporterPesewas = grossPesewas - platformFeePesewas`.

Use a 64-bit integer column. **NUMERIC / DECIMAL** is acceptable; **FLOAT / REAL / DOUBLE** is a defect.

3. Domain model

3.1 Incident categories

23 categories in 6 groups. The groups exist for interface organisation; the category is what is stored and routed on.

```
export const INCIDENT_CATEGORIES = [  
  // Emergency  
  'fire', 'accident', 'flood', 'weather', 'health',  
  // Crime and safety  
  'crime', 'disorder', 'protest',  
  // Public services  
  'utility', 'water', 'sanitation', 'road', 'transport', 'infrastructure', 'education',  
  // Governance  
  'corruption', 'election', 'chieftaincy', 'land',  
  // Environment  
  'galamsey', 'environment', 'wildlife',  
  // Catch-all  
  'other',  
] as const;  
  
export type IncidentCategory = (typeof INCIDENT_CATEGORIES)[number];
```

galamsey is illegal small-scale mining. **chieftaincy** covers traditional-authority disputes. Both are specific to the Ghanaian context and are not synonyms for **crime** or **disorder**.

3.2 Severity

The reporter's own account of urgency. **A claim, not a dispatch.**

```
export type Severity = 'observation' | 'concern' | 'urgent' | 'emergency';
```

VALUE	MEANING	WEIGHT	WARNS ABOUT 112
observation	Worth recording. Nobody at risk.	1	no
concern	Should be looked at. Worse if ignored.	2	no
urgent	Needs attention today. Property or services affected.	3	no
emergency	People are in danger right now.	4	yes

Ghana's emergency number is **112**. Dawuro is not an emergency service and must never be mistaken for one — selecting **emergency** shows the reporter a warning to call 112 before the report can be sent. The backend **MUST**

NOT treat `emergency` as a dispatch trigger to any emergency service. It affects queue ordering and the acknowledgement target (§11) and nothing else.

3.3 Consent flags and handling requirements

What the footage contains, as the reporter describes it. These drive **handling**, not sorting.

```
export interface ConsentFlags {
  subjectsConsented: boolean;    // identifiable people who agreed
  publicPlace: boolean;         // consent not ordinarily required
  containsMinors: boolean;      // children identifiable
  distressing: boolean;         // injury, death, distressing to view
  showsPrivateProperty: boolean; // a private home, interior or entrance
}

export type HandlingRequirement =
  | 'redact_before_publication'
  | 'viewer_warning'
  | 'restrict_location'
  | 'editorial_review_required';
```

Handling is derived, never chosen. A reporter ticking "there are children in it" is describing the footage; deciding it therefore cannot be published unredacted is the platform's job. The server **MUST** compute this and **MUST NOT** accept a client-supplied handling list.

```
export function handlingRequirements(consent: ConsentFlags): HandlingRequirement[] {
  const required: HandlingRequirement[] = [];

  if (consent.containsMinors) {
    required.push('redact_before_publication', 'editorial_review_required');
  }
  if (consent.distressing) {
    required.push('viewer_warning', 'editorial_review_required');
  }
  if (consent.showsPrivateProperty) {
    required.push('restrict_location');
    // Identifiable people at a private address who did not agree to be filmed
    // is the case most likely to cause real harm.
    if (!consent.subjectsConsented && !consent.publicPlace) {
      required.push('redact_before_publication');
    }
  }

  return [...new Set(required)];
}
```

`redact_before_publication` is a hard block on the public feed regardless of how well corroborated a report is. See §14.

3.4 Media, location, time precision

```
export type MediaKind = 'photo' | 'video' | 'audio';
```

Audio is a **first-class** report type, not a lesser one. Describing an incident from somewhere safe carries none of the risk of filming it, and it is the only mode that works in the dark or in a crowd. It is priced accordingly (\$10.2).

```
export type LocationConfidence = 'high' | 'low'; // 'low' = reporter used the reduced-accuracy escape hatch
export type TimePrecision = 'exact' | 'date_only' | 'hidden';

export interface IncidentMedia {
  kind: MediaKind;
  url: string;
  posterUrl: string; // still frame for video; shown while the player warms up
  width: number;
  height: number;
  durationMs?: number;
  byteSize?: number;
}

export interface PublicLocation {
  latitude: number | null;
  longitude: number | null;
  label: string | null; // "Kaneshie, Accra"
  confidence: LocationConfidence;
}

export interface PreciseLocation extends PublicLocation {
  accuracyM: number;
  altitude: number | null;
  heading: number | null;
  speed: number | null;
  isMocked: boolean;
}
```

`PreciseLocation` is **author-only** and **MUST NEVER** appear on a public endpoint.

`TimePrecision` is explicit rather than inferred, and this matters: when `showTime` is false the server truncates `capturedAtIso` to midnight UTC. A client inferring precision from the value alone could not distinguish a suppressed time from an incident that genuinely happened at 00:00, and would either leak a real midnight capture or mislabel a hidden one.

3.5 Display flags

```
export interface DisplayFlags {
  showLocation: boolean;
  showDate: boolean;
  showTime: boolean;
}
```

Set by the reporter at review, editable later by the account owner. These are **display** flags, not storage flags: the server retains the true values for vetting and lawful process, and MUST null them out on public responses.

Both layers enforce this independently. Client-only hiding leaks coordinates into every phone's cache; server-only hiding breaks the moment one client has a bug. Implement it server-side regardless of what the client does.

FLAG FALSE	SERVER MUST NULL / TRUNCATE
<code>showLocation</code>	<code>location.latitude</code> , <code>location.longitude</code> , <code>location.label</code>
<code>showDate</code>	<code>capturedAtIso</code> → <code>null</code> , <code>capturedAtPrecision</code> → <code>'hidden'</code>
<code>showTime</code>	<code>capturedAtIso</code> → midnight UTC of that date, <code>capturedAtPrecision</code> → <code>'date_only'</code>

3.6 The incident

```
export type VettingState = 'pending_review' | 'published' | 'rejected' | 'restricted';

export type Publisher =
  | { kind: 'anonymous' }
  | { kind: 'user'; id: string; displayName: string; avatarUrl: string | null };

export interface IncidentCounts { reactions: number; comments: number }

export interface Incident {
  id: string;
  reportId: string; // see §3.7
  category: IncidentCategory;
  description: string;
  vettingState: VettingState;
  publishedAt: string;
  media: IncidentMedia;
  location: PublicLocation;
  capturedAtIso: string | null; // null / truncated per display flags
  capturedAtPrecision: TimePrecision;
  publisher: Publisher;
  assurance: AssuranceClass; // a technical fact – §4.1
  verification: VerificationState; // an editorial judgement – §4.2
  severity: Severity;
  landmark: string | null;
  handling: HandlingRequirement[]; // derived, §3.3
  counts: IncidentCounts;
  viewerHasReacted: boolean;
  distanceM?: number; // only when the query passed `near`
}

/** The author's own view – adds everything the public must not see. */
export interface AuthoredIncident extends Omit<Incident, 'location'> {
  location: PreciseLocation;
  displayFlags: DisplayFlags;
  isAnonymous: boolean;
  rejectionReason: string | null;
  clientId: string; // mirrors the device's local row id
}
```

`assurance` and `verification` are separate fields and MUST remain separate. See §4.

3.7 Report ID

`id` is for machines. `reportId` is the human reference **burned into the video frame at capture**, read down a phone line, and typed into the public verification page. Both clients generate it with this exact function and the server MUST produce identical output.

```
// Excludes every vowel, so an id can never spell a word – nobody should have
// to read an unfortunate code back to a call centre. Also drops the pairs
// people confuse in a low-resolution overlay burned into video:
// 0/O, 1/I/L, 2/Z, 5/S, 8/B.
const ID_ALPHABET = '34679CDEFGHJKMNPQRTVWXY';

export function formatReportId(seed: string): string {
  let hash = 2166136261; // FNV-1a 32-bit offset basis
  for (let i = 0; i < seed.length; i += 1) {
    hash ^= seed.charCodeAt(i);
    hash = Math.imul(hash, 16777619); // FNV prime, 32-bit wraparound
  }

  let value = hash >>> 0;
  let out = '';
  for (let i = 0; i < 6; i += 1) {
    out += ID_ALPHABET[value % ID_ALPHABET.length];
    value = Math.floor(value / ID_ALPHABET.length);
  }
  return `DW-${out.slice(0, 3)}-${out.slice(3)}`;
}
```

`Math.imul` is a 32-bit signed multiply with wraparound. In other languages, multiply as unsigned 32-bit and mask to `0xFFFFFFFF`. Test vectors are in §15.

Format: `DW-XXX-XXX`. Lookups MUST be case-insensitive.

This is a hash, so collisions are possible. For production, either persist a generated `reportId` with a uniqueness constraint and re-seed on collision, or switch to a sequence. Do not assume the hash alone is unique across millions of rows.

4. The trust model

This is the core of the product. Read it completely before implementing anything that touches it.

Dawuro answers two separate questions about every piece of media, and **must never allow them to merge**:

- **Assurance class** — *was this captured through the trusted app, and is the stored file unchanged?* A technical fact a machine establishes.
- **Verification state** — *is the claim about what it depicts substantively true?* An editorial judgement no amount of cryptography can produce.

A file can be Class A — perfect integrity, valid signature, clock and GPS agreeing — and still show a staged event. **Saying "verified" because a hash checked out is the single most damaging mistake this platform could make.** The two are modelled apart, stored apart, and the language permitted for each is enumerated rather than left to whoever writes the interface.

4.1 Assurance class

```
export type AssuranceClass = 'A' | 'B' | 'C' | 'D';
```

CLASS	LABEL	THE ONLY PHRASE THAT MAY BE SHOWN	EXPEDITED	USABLE ALONE
A	Trusted capture	<i>Assured capture integrity</i>	yes	yes
B	Trusted capture with flags	<i>Capture integrity partially verified</i>	no	yes
C	External upload	<i>Origin not technically verified</i>	no	no
D	Institutional evidence	<i>Authorised institutional capture</i>	yes	yes

Class C is usable **only as a lead** unless independently corroborated. That is a rule, not a guideline (§4.5).

The class is derived from facts a machine can establish. Nothing here is a judgement, which is what keeps the classification reproducible.

```
export interface CaptureFacts {
  capturedInApp: boolean;           // false for anything imported
  integritySignatureValid: boolean; // stored file still matches the capture signature
  timeCheckPassed: boolean;        // device clock agreed with server time within tolerance
  locationCheckPassed: boolean;    // a fix of usable accuracy was held throughout
  deviceCheckPassed: boolean;      // not rooted, emulated or otherwise flagged
  originalPreserved: boolean;      // the untouched original is still held
  institutionalCapture: boolean;    // accredited officer under institutional controls
}

export function assuranceClass(facts: CaptureFacts): AssuranceClass {
  // Order is deliberate and MUST NOT be rearranged. An import can never be
  // rescued into a trusted class by other checks passing – otherwise a gallery
  // file with a plausible clock would classify as institutional evidence.
  if (!facts.capturedInApp) return 'C';
  if (facts.institutionalCapture) return 'D';

  const allChecksPassed =
    facts.integritySignatureValid &&
    facts.timeCheckPassed &&
    facts.locationCheckPassed &&
    facts.deviceCheckPassed &&
    facts.originalPreserved;

  return allChecksPassed ? 'A' : 'B';
}
```

The server computes this. It MUST NOT accept an assurance class from a client. The client displays what the server returns.

4.2 Verification state

```
export type VerificationState =
  | 'received_unreviewed'
  | 'integrity_passed'
  | 'integrity_flagged'
  | 'corroboration_in_progress'
  | 'verified_high_confidence'
  | 'verified_in_part'
  | 'disputed'
  | 'rejected';
```

STATE	MEANING	PUBLISH	LICENSE	MAY SAY "VERIFIED"
received_unreviewed	Exists; no editorial conclusion	X	X	X
integrity_passed	Capture and file controls passed	X	✓	X
integrity_flagged	One or more technical checks need review	X	X	X
corroboration_in_progress	Editorial is checking source and claim	X	✓	X
verified_high_confidence	Sufficient corroboration for the stated facts	✓	✓	✓
verified_in_part	Some elements established, others unresolved	✓	✓	✓
disputed	Credible contradictory information exists	X	X	X
rejected	Fabricated, harmful, illegal, unsafe, compromised	X	X	X

Permitted representation. Each state carries the one sentence the interface is allowed to show. This is legal exposure, not a copy decision — "capture integrity verified" and "event verified" are different claims, and a product that lets them blur will eventually publish the second while having established only the first. The API MUST return this string and the clients MUST render it verbatim.

STATE	PERMITTED REPRESENTATION
received_unreviewed	Never describe as verified.
integrity_passed	Capture integrity verified — not event verified.
integrity_flagged	Show the specific flag to authorised reviewers only.
corroboration_in_progress	May be shared only as a lead, where policy allows.
verified_high_confidence	Publish or license, stating the basis and its limitations.
verified_in_part	State precisely what is and is not verified.
disputed	Attach the update history; do not syndicate stale copies.
rejected	Restricted audit retention only.

4.3 Allowed transitions

The server MUST reject any transition not in this table, with `403 FORBIDDEN`.

FROM	MAY MOVE TO
received_unreviewed	integrity_passed , integrity_flagged , rejected
integrity_passed	corroboration_in_progress , integrity_flagged , rejected
integrity_flagged	integrity_passed , corroboration_in_progress , rejected
corroboration_in_progress	verified_high_confidence , verified_in_part , disputed , rejected
verified_high_confidence	verified_in_part , disputed , rejected
verified_in_part	verified_high_confidence , disputed , rejected
disputed	corroboration_in_progress , verified_in_part , rejected
rejected	nothing — terminal

Two rules here are structural rather than procedural:

1. **Nothing reaches a verified state without passing through `corroboration_in_progress`**. Verification is the act of corroborating. Allowing the jump would let one click turn an unreviewed submission into a published fact.
2. **`rejected` is terminal**. Reinstating something judged fabricated or unsafe must be a new submission with a new record, not an edit that quietly erases the judgement.

4.4 Corroboration

Moving into a verified state requires corroboration strength above a threshold, and independence is checked **before** strength.

```

const REQUIRED_STRENGTH = {
  verified_high_confidence: 0.5,
  verified_in_part: 0.15,
};

// Independence first: a Class C upload corroborated only by things that share
// its source is not corroborated at all.
if (!ASSURANCE_META[assurance].usableAlone && !hasIndependentCorroboration(record)) {
  return 'needs_independent_corroboration';
}
if (corroborationStrength(record) < required) {
  return 'insufficient_corroboration';
}

```

Corroboration checks are recorded individually against a report, each with a weight and a source.

`corroborationStrength` is the normalised sum of passed checks. An independent check is one whose source is not the reporter and not another party the reporter supplied.

4.5 The publish and license gates

```

export function canPublishReport(
  state: VerificationState,
  assurance: AssuranceClass,
  independentlyCorroborated = false,
): boolean {
  if (!VERIFICATION_META[state].publishable) return false;
  // The class gates the state, not the other way round. An external file an
  // editor marked verified is still only usable as corroborated material.
  if (!ASSURANCE_META[assurance].usableAlone && !independentlyCorroborated) return false;
  return true;
}

export function canLicenseReport(state: VerificationState): boolean {
  return VERIFICATION_META[state].licensable;
}

```

The server MUST enforce both at the point of publication and licensing, not merely when the state changes.

4.6 Mapping to the legacy public state

The public feed still renders the coarse four-state value. Everything collapses toward the safer answer — anything not publishable reads as pending rather than published.

```

export function vettingStateFor(state: VerificationState): VettingState {
  if (state === 'rejected') return 'rejected';
  if (state === 'disputed' || state === 'integrity_flagged') return 'restricted';
  if (VERIFICATION_META[state].publishable) return 'published';
  return 'pending_review';
}

```

5. Capture and upload

5.1 The protocol

A 60-second video over Ghanaian mobile data **will** drop mid-flight. Uploads are resumable and chunked, in three phases.

PHASE	CALL	CARRIES
1	<code>POST /incidents</code>	Metadata only. Returns <code>uploadId</code> and the chunk plan.
2	<code>PUT /uploads/{id}/chunks/{n}</code>	Raw bytes. Repeatable, order-independent.
3	<code>POST /uploads/{id}/complete</code>	Nothing. Server assembles, verifies hash, enqueues.

If the app dies mid-upload, the client calls `GET /uploads/{id}` on next launch to learn which chunks you already hold and resumes. **It does not restart from zero.**

5.2 `POST /devices` — register a device (*no auth*)

```
{ "platform": "ios", "appVersion": "1.0.0", "installId": "e3a1..." }
```

Returns `201` with `{ "deviceToken": "...", "deviceId": "dev_..." }`.

5.3 `POST /incidents` — initialise (*auth: device or user*)

Metadata only, no bytes. Requires `Idempotency-Key`.

```

{
  "clientId": "e3a1...",
  "category": "flood",
  "description": "Culvert blocked, road impassable at the junction.",
  "isAnonymous": true,
  "displayFlags": { "showLocation": true, "showDate": true, "showTime": false },

  "severity": "urgent",
  "landmark": "Kaneshie First Light",
  "consent": {
    "subjectsConsented": false,
    "publicPlace": true,
    "containsMinors": false,
    "distressing": false,
    "showsPrivateProperty": false
  },
  "requestedAction": "Clear the culvert before the next rain.",
  "observedSubjects": null,

  "destination": "marketplace",
  "requestedBusinessIds": [],

  "location": {
    "latitude": 5.603717, "longitude": -0.186964,
    "accuracyM": 8.4, "altitude": 61.2, "heading": 142.0, "speed": 0.0,
    "confidence": "high", "isMocked": false
  },

  "capturedAtIso": "2026-08-27T14:31:00.000Z",
  "capturedAtUtcOffsetMinutes": 0,
  "deviceUptimeMs": 184523110,

  "media": {
    "kind": "video", "mimeType": "video/mp4",
    "byteSize": 8421376, "durationMs": 24500,
    "width": 1080, "height": 1920,
    "sha256": "9f86d081884c7d65..."
  }
}

```

Returns **201**:

```

{
  "incidentId": "inc_01JBX7Q2K9",
  "reportId": "DW-WQD-JW4",
  "uploadId": "upl_01JBX7Q2K9",
  "chunkSizeBytes": 5242880,
  "chunkCount": 2,
  "expiresAt": "2026-08-28T14:31:00.000Z"
}

```

`clientId` is the device's local row id. **Return it on the resulting incident** so the outbox can reconcile without relying on ordering.

`sha256` is of the complete media file. Verify at `complete` and reject with `MEDIA_HASH_MISMATCH` if it differs — that is the client's signal to re-upload rather than the platform silently publishing corrupt footage.

The client proposes `chunkSizeBytes = 5 MiB`; **your response is authoritative** and the client uses whatever you return.

`mimeType` for audio is `audio/m4a`, and `width` / `height` are `null`.

5.4 What the server **MUST** establish at submission

This is what the mobile app cannot do for itself and what the whole trust model rests on. **None of it is currently implemented anywhere.**

Server-side integrity signature. On `complete`, hash the assembled file and sign the tuple `(sha256, incidentId, serverReceivedAtIso, deviceId)` with a server-held key. Store the signature. This is what `integritySignatureValid` checks against later. The device cannot be trusted to sign for itself — a compromised device signs whatever it likes.

Server-time comparison. Compare `capturedAtIso` against the time the server received the initialisation, accounting for `deviceUptimeMs` and `capturedAtUtcOffsetMinutes`. Set `timeCheckPassed` false when the gap is implausible for the upload path. A device clock set to last year is the cheapest way to fake a capture time, and `capturedAtIso` is otherwise entirely device-controlled. **Suggested tolerance: 5 minutes of drift plus the observed upload delay; confirm with the team (\$16).**

Location plausibility. Set `locationCheckPassed` false when `accuracyM` exceeds the category limit, `isMocked` is true, or the fix is inconsistent with the device's recent history.

Device integrity. Set `deviceCheckPassed` from attestation (Play Integrity / App Attest) where available.

Original preservation. Store the original bytes immutably and separately from any derived or transcoded copy. `originalPreserved` is false the moment you cannot produce the file the device sent.

Malware and duplicate screening. Scan on `complete`. Perceptual-hash against existing media to catch a file resubmitted as a fresh incident.

Only once these are set does `assuranceClass()` (\$4.1) run. Until it has, the report sits at `received_unreviewed`.

5.5 `PUT /uploads/{uploadId}/chunks/{index}` (*auth: same as init*)

`index` is **0-based**. Body is `application/octet-stream`, raw bytes, no multipart wrapper.

Returns `200` with `{ "index": 0, "received": true, "bytesReceived": 5242880 }`.

MUST be **idempotent** — re-sending a chunk you already hold returns `200`, not an error. Chunks may arrive in any order; the client sends sequentially but retries reorder them.

5.6 `POST /uploads/{uploadId}/complete` (*auth: same as init*)

Empty body. Assembles, verifies the hash, runs \$5.4, enqueues for vetting. Returns `200` with the full incident in its author shape.

Only after this succeeds does the client delete its local media file. **It keeps the metadata row** so the user retains a personal history.

5.7 `GET /uploads/{uploadId}` — resume (*auth: same as init*)

```
{
  "uploadId": "upl_01JBX7Q2K9",
  "chunkSizeBytes": 5242880,
  "chunkCount": 2,
  "receivedChunks": [0],
  "expiresAt": "2026-08-28T14:31:00.000Z"
}
```

Expired uploads return `410 UPLOAD_EXPIRED` and the client restarts from `POST /incidents`.

6. Routing — which institutions receive a report

Reports reach institutions on their own. A reporter picks a category and optionally names organisations; everything else is matching. Requiring a human to touch every submission does not survive contact with volume.

The platform operator's desk sits *above* this: they see every submission and what it was auto-routed to, and can add or remove recipients. **Oversight and override, not a gate.**

```
export type SubmissionDestination = 'public' | 'marketplace' | 'directed' | 'both';
```

A reporter chooses this at review time and it changes everything downstream: who sees the report, whether it earns, and who reviews it.

DESTINATION	PUBLIC FEED	OFFERED TO INSTITUTIONS	EARNS
<code>public</code>	✓	X	X — visibility is the reward
<code>marketplace</code>	X	✓	✓ if licensed
<code>directed</code>	X	only those named	✓ if licensed, ×1.25
<code>both</code>	✓	✓	✓ if licensed

`marketplace` and `both` route identically; they differ only in whether the report also appears publicly.

```
export interface RoutableSubmission {
  category: IncidentCategory;
  destination: SubmissionDestination;
  requestedBusinessIds: string[];
  location: { latitude: number; longitude: number } | null; // null when suppressed
}

export interface BusinessWatchArea {
  businessId: string;
  latitude: number; longitude: number; radiusM: number;
}

export type RouteReason = 'requested' | 'interest_match' | 'in_watch_area' | 'has_allowance';

export interface RouteMatch {
  businessId: string;
  score: number; // higher is a better fit; drives operator review order
  reasons: RouteReason[];
}
```

The algorithm

```
export function autoRoute(
  submission: RoutableSubmission,
  businesses: readonly BusinessAccount[],
  watchAreas: readonly BusinessWatchArea[] = [],
): RouteMatch[] {
  // A public-only report is never offered to businesses. Routing it would
  // quietly turn a free contribution into a commercial one.
  if (submission.destination === 'public') return [];

  const matches: RouteMatch[] = [];

  for (const business of businesses) {
    if (!canReceive(business)) continue; // subscription must be active or trialing

    const reasons: RouteReason[] = [];
    let score = 0;

    const requested = submission.requestedBusinessIds.includes(business.id);
    if (requested) {
      reasons.push('requested');
      score += 100; // an explicit request outranks every heuristic
    }

    if (business.interests.includes(submission.category)) {
      reasons.push('interest_match');
      score += 40;
    }

    // Geography applies only when a location was published. A suppressed
    // location must not silently exclude anyone – it means we cannot tell,
    // not that the answer is no.
    if (submission.location) {
      const areas = watchAreas.filter((a) => a.businessId === business.id);
      const inside = areas.some((area) => distanceMetres(submission.location!, area) <= area.radiusM);
      if (inside) {
        reasons.push('in_watch_area');
        score += 30;
      } else if (areas.length > 0 && !requested) {
        continue; // they drew a boundary; respect it
      }
    }

    if (business.reportsUsedThisPeriod < 1_000) {
      reasons.push('has_allowance');
      score += 10;
    }

    // A directed submission goes only to the named institutions.
    if (submission.destination === 'directed' && !requested) continue;

    // Everything else needs a positive reason. Matching on "has allowance"
    // alone would send a wildlife report to an insurer purely because they
    // had budget left.
```

```
    if (!requested && !reasons.includes('interest_match') && !reasons.includes('in_watch_area')) {
      continue;
    }

    matches.push({ businessId: business.id, score, reasons });
  }

  return matches.sort((a, b) => b.score - a.score);
}
```

Distance is haversine on a sphere of radius **6,371,008.8 m**.

An empty result is a legitimate outcome, not an error. A report nobody declared an interest in surfaces on the operator's desk rather than being forced onto an arbitrary recipient.

```
export function needsReview(submission: RoutableSubmission, matches: RouteMatch[]): boolean {
  if (matches.length === 0) return true; // an operator may know a recipient the rules do
  const matched = new Set(matches.map((m) => m.businessId));
  return submission.requestedBusinessIds.some((id) => !matched.has(id));
}
```

Routing must run in real time on submission, not on a nightly batch. A flood report routed tomorrow is worthless.

7. Assignment — which employee handles it

Once a report reaches an institution, it is assigned to a person. Two stages: hard gates, then a weighted score. Location alone is not enough — the nearest person may be off duty, at capacity, or unqualified.

7.1 Hard gates

Any of these blocks assignment outright.

```

export type AssignmentBlock =
  | 'inactive' | 'on_leave' | 'off_duty'
  | 'at_capacity' | 'not_qualified' | 'outside_jurisdiction';

export function blockingReason(employee, incident, branch): AssignmentBlock | null {
  if (!employee.active) return 'inactive';
  if (employee.shiftStatus === 'on_leave') return 'on_leave';
  if (employee.shiftStatus === 'off_duty') return 'off_duty';
  if (employee.openAssignments >= employee.maxConcurrentAssignments) return 'at_capacity';

  // An empty specialisation list means generalist – eligible for anything
  // rather than nothing. Empty far more often means "not filled in yet" than
  // "qualified for no incident on earth", and defaulting to exclusion would
  // quietly empty an organisation's routing.
  if (employee.specialisations.length > 0 && !employee.specialisations.includes(incident.category))
    return 'not_qualified';

  if (branch && incident.location) {
    if (haversineMetres(branch.location, incident.location) > branch.jurisdictionRadiusM)
      return 'outside_jurisdiction';
  }
  return null;
}

```

7.2 Weighted score

Applied to everyone who passes the gates. Highest wins.

FACTOR	WEIGHT	AWARDED WHEN
specialisation	40	The incident category is in their specialisations
proximity	30	Scaled by nearness of their last known position
fieldDuty	25	They hold a duty suited to the category
capacity	20	Scaled by the share of their capacity still free
seniorUrgent	20	Senior staff on urgent or emergency reports
branchArea	15	The incident falls inside their branch's jurisdiction
reliability	15	Scaled by acknowledgement rate
language	10	They speak a language the reporter used

acknowledgementRate is **reliability, not productivity**. Someone who never acknowledges should stop receiving urgent work regardless of how close they are.

lastKnownLocation is null for office staff and for anyone who has not consented, and **MUST NEVER be inferred from a branch address**. A desk is not a person, and pretending otherwise sends a patrol to a building.

8. Organisations

An organisation is rarely one office. A metropolitan assembly has sub-metros, a utility has district depots, a media house has regional bureaux — and an incident that belongs to one does not belong to the others.

8.1 The account

```
export type BusinessSector =
  | 'government' | 'media' | 'utility' | 'insurance' | 'ngo' | 'research' | 'other';

export interface BusinessAccount {
  id: string;
  name: string;
  sector: BusinessSector;
  verified: boolean;
  tier: SubscriptionTier;
  subscriptionStatus: 'trialing' | 'active' | 'past_due' | 'cancelled';
  renewsAtIso: string;
  seatsUsed: number;
  reportsUsedThisPeriod: number;
  interests: IncidentCategory[];
  logoUrl: string | null;
}
```

8.2 Branches

```
export interface Branch {
  id: string;
  businessId: string;
  name: string;
  location: LatLng;
  jurisdictionRadiusM: number;
  areaLabel: string;
}
```

A circle is a crude stand-in for a real administrative boundary, which is a polygon. It is deliberately crude: the shape of these boundaries is each organisation's decision, and guessing at polygons would bake in borders nobody agreed to. Scoring treats it as a hard limit, so replacing it with real geometry later changes one function.

Set radii realistically. A metropolitan assembly covering 7 km will route almost nothing.

8.3 Employees

```
export type EmployeeDuty =
  | 'field_response' | 'dispatch' | 'investigation'
  | 'inspection' | 'media' | 'community_liaison' | 'admin';

export type ShiftStatus = 'on_duty' | 'off_duty' | 'on_leave';
export type WorkingLanguage = 'en' | 'twi' | 'ga' | 'ewe' | 'dagbani' | 'hausa';

export interface Employee {
  id: string;
  businessId: string;
  branchId: string | null;           // null = unassigned, still routable org-wide
  displayName: string;
  email: string;
  phone: string | null;

  role: OrgRole;                    // software permissions
  duties: EmployeeDuty[];           // field responsibilities
  specialisations: IncidentCategory[]; // empty = generalist
  languages: WorkingLanguage[];

  shiftStatus: ShiftStatus;
  openAssignments: number;
  maxConcurrentAssignments: number;

  lastKnownLocation: LatLng | null;
  lastSeenAtIso: string | null;
  acknowledgementRate: number;      // 0..1

  joinedAtIso: string;
  active: boolean;
}
```

`EmployeeDuty` is deliberately separate from `OrgRole`. A dispatcher and a field responder may both be `analyst` in permission terms while being completely different people to send to a fire. Collapsing the two would mean granting software access in order to describe someone's job, which is how permission systems rot.

8.4 Roles and capabilities

```
export type OrgRole = 'owner' | 'admin' | 'analyst' | 'dispatcher' | 'viewer';
```

CAPABILITY	VIEWER	ANALYST	DISPATCHER	ADMIN	OWNER
view_reports	✓	✓	✓	✓	✓
export_data		✓		✓	✓
trriage_reports			✓	✓	✓
dispatch_units			✓	✓	✓
manage_queries				✓	✓
manage_members				✓	✓
manage_org					✓

`dispatcher` sits above `analyst` deliberately: sending a patrol car to a location is a heavier action than reading a trend chart, even though the dispatcher sees less data.

Three rules the server MUST enforce:

- An admin cannot change or remove an **owner**. Without this an admin demotes the owner and takes the organisation.
- **The last owner can never be removed.** An organisation with no owner has nobody who can restore access, and recovery becomes a support ticket.
- Only an owner can create another owner.

These are advisory in the client and authoritative in the server. The console hides buttons as a courtesy; hiding a button secures nothing.

8.5 Employees joining

```
export type MembershipStatus = 'pending' | 'accepted' | 'rejected';

export interface MembershipRequest {
  id: string;
  businessId: string;
  requestedBranchId: string | null;
  displayName: string;
  email: string;
  phone: string | null;
  statedRole: string; // what they say they do, pending confirmation
  signUpMethod: 'password' | 'google';
  emailVerified: boolean;
  requestedAtIso: string;
  status: MembershipStatus;
  note: string | null;
}
```

Employees sign themselves up and name their employer; **the organisation decides whether that is true**. Self-service in both directions would let anyone claim to work for the Electoral Commission and start receiving election footage. Acceptance is always an act by someone already inside who holds `manage_members`.

8.6 Internal submissions

A report an employee sent straight to their own organisation. These bypass the marketplace entirely — the organisation already employs the person, so there is nothing to license and no commission to pay.

```
export interface InternalSubmission {
  id: string;
  businessId: string;
  employeeId: string;
  employeeName: string;
  branchId: string | null;
  category: IncidentCategory;
  summary: string;
  posterUrl: string;
  locationLabel: string | null;
  location: LatLng | null;
  capturedAtIso: string;
  submittedAtIso: string;
  status: 'new' | 'assigned' | 'resolved' | 'dismissed';
  assignedToEmployeeId: string | null;
}
```

Internal submissions are never anonymous. An internal report is only actionable if you know which member of staff filed it.

8.7 Invites and affiliations

Organisations send links to employees, agents and affiliates.

```
export type InviteKind = 'employee' | 'agent' | 'affiliate_org' | 'reporter';
export type InviteProblem = 'revoked' | 'expired' | 'exhausted';

export function inviteProblem(invite: Invite, nowIso: string): InviteProblem | null {
  if (invite.revokedAtIso) return 'revoked';
  if (Date.parse(invite.expiresAtIso) <= Date.parse(nowIso)) return 'expired';
  if (invite.maxUses !== null && invite.usedCount >= invite.maxUses) return 'exhausted';
  return null;
}
```

An organisation may be affiliated to another organisation. Affiliation is directional and **MUST NOT** be allowed to form a cycle — `canAffiliate` rejects an affiliation that would make an organisation its own ancestor.

An individual reporter declares an institution they are affiliated with, or declares themselves independent. Both are first-class; independence is not a missing value.

9. Institutional onboarding

Registration says who is asking. Onboarding is the evidence, and it is a different shape of thing: a sequence of steps an applicant works through at their own pace, each one submitted and then reviewed by a platform administrator before any of it counts.

Two rules give the flow its shape:

1. **The applicant submits; the platform approves.** Completing a step never approves it. An organisation that could mark its own evidence acceptable is an organisation that has not been checked.
2. **Approval is per step, and the final decision is gated on all of them.** Reviewing an application as one blob means a reviewer accepts everything or rejects everything, and the applicant is told "declined" with no idea which document was wrong.

9.1 The flow

```
register (3 steps: organisation → interests → plan)
  ↓ account created immediately
onboarding (4 evidence steps, each reviewed separately)
  ↓ all steps approved + screening clear
platform administrator approves → institution can receive footage
```

Registration collects what an officer can answer from memory in under a minute. **Onboarding collects evidence and only evidence** — categories and the plan are chosen at registration and are deliberately not asked again, because asking twice makes the first pass look pointless and the second look like the form forgot.

9.2 Steps

```
export type OnboardingStepId = 'organisation' | 'officer' | 'coverage' | 'documents';

export type StepStatus =
  | 'not_started' // not opened
  | 'in_progress' // opened, partly filled, not sent
  | 'submitted'   // sent for review; the applicant can no longer edit it
  | 'approved'    // a reviewer accepted it
  | 'rejected';   // a reviewer sent it back, with a reason
```

STEP	COLLECTS	DOCUMENTS
<code>organisation</code>	Legal name, sector, registration number	<code>business_registration</code> , <code>tax_identification</code>
<code>officer</code>	The person who signs for this account	<code>officer_id</code> , <code>authorisation_letter</code>
<code>coverage</code>	Where you operate — decides which incidents can reach your staff	<code>premises_proof</code>
<code>documents</code>	Everything attached, and anything still missing	—

9.3 Documents

```
export type DocumentId =
  | 'business_registration' | 'tax_identification'
  | 'officer_id' | 'authorisation_letter'
  | 'premises_proof' | 'utility_bill' | 'lease_agreement';
```

DOCUMENT	REQUIRED	NOTES
<code>business_registration</code>	✓	Certificate of incorporation, or equivalent for a public body
<code>tax_identification</code>	✓	TIN certificate
<code>officer_id</code>	✓	Ghana Card, passport or driver's licence
<code>authorisation_letter</code>	✓	On letterhead, confirming this person may act for the organisation
<code>premises_proof</code>	✓	Satisfied by a lease or a utility bill
<code>lease_agreement</code>		Alternative for <code>premises_proof</code>
<code>utility_bill</code>		Alternative for <code>premises_proof</code> ; within the last three months

Alternatives. A lease and a utility bill both prove occupancy, and demanding both is how an application stalls on paperwork that adds nothing. Any one member of an `alternativeGroup` satisfies the requirement.

```
export function documentSatisfied(application, id: DocumentId): boolean {
  const requirement = DOCUMENT_REQUIREMENTS[id];
  const uploaded = new Set(application.documents.map((d) => d.id));

  if (uploaded.has(id)) return true;
  if (!requirement.alternativeGroup) return false;

  return Object.values(DOCUMENT_REQUIREMENTS).some(
    (other) => other.alternativeGroup === requirement.alternativeGroup && uploaded.has(other.id),
  );
}
```

9.4 The gates

```
export type SubmitProblem = 'steps_outstanding' | 'documents_missing' | 'already_submitted';

export type ApprovalProblem =
  | 'not_submitted' | 'steps_not_approved'
  | 'screening_not_run' | 'screening_not_clear' | 'already_approved';

export function approvalProblem(application): ApprovalProblem | null {
  if (application.approvedAtIso) return 'already_approved';
  if (!application.submittedAtIso) return 'not_submitted';

  const unapproved = ONBOARDING_STEPS.some(
    (meta) => stepState(application, meta.id).status !== 'approved',
  );
  if (unapproved) return 'steps_not_approved';

  if (application.screeningRunAtIso === null) return 'screening_not_run';
  if (application.screeningClear !== true) return 'screening_not_clear';

  return null;
}
```

Screening is a sanctions and adverse-media check on both the organisation and its named officer. It **MUST** have been run and **MUST** have come back clear.

Approval is what grants an organisation access to footage of the public, so it is the one decision in the product that no single click should be able to reach.

A rejected step jumps the queue in `nextStepFor`, because everything after it may depend on the thing that was wrong, and leaving it buried is how an application sits untouched for a fortnight.

Reference format: `ONB-ORG-000042`, quoted in every email about the application.

10. Money

10.1 Subscription plans

Every figure is integer pesewas. 100 pesewas = GHS 1.

	BASIC	STANDARD	ENTERPRISE
Billing period	monthly	monthly	annual
Fee	45,000 (GHS 450/mo)	180,000 (GHS 1,800/mo)	2,400,000 (GHS 24,000/yr)
Per download	2,000 (GHS 20)	1,200 (GHS 12)	null — unlimited
Seats	3	12	50
Concurrent surveys	1	5	25
Can direct-request	no	yes	yes

```
export interface SubscriptionPlan {
  tier: SubscriptionTier;           // 'basic' | 'standard' | 'enterprise'
  billingPeriod: BillingPeriod;     // 'monthly' | 'annual'
  feePesewas: number;
  perDownloadPesewas: number | null; // null means unlimited
  seats: number;
  concurrentSurveys: number;
  canDirectRequest: boolean;
}
```

```
export function isUnlimited(plan) { return plan.perDownloadPesewas === null; }
export function downloadCharge(plan) { return plan.perDownloadPesewas ?? 0; }

export function periodCost(plan, downloads: number): number {
  const taken = Math.max(0, Math.floor(downloads)); // negative/fractional is a programming error
  return plan.feePesewas + downloadCharge(plan) * taken;
}

export function annualCost(plan, downloadsPerYear: number): number {
  const taken = Math.max(0, Math.floor(downloadsPerYear));
  const periods = plan.billingPeriod === 'annual' ? 1 : 12;
  return plan.feePesewas * periods + downloadCharge(plan) * taken;
}

/** Downloads per year at which unlimited becomes cheaper. Null when metered never loses. */
export function breakEvenDownloads(metered, unlimited): number | null {
  const perDownload = metered.perDownloadPesewas;
  if (perDownload === null || perDownload <= 0) return null;

  const meteredAnnualFee = metered.feePesewas * (metered.billingPeriod === 'annual' ? 1 : 12);
  const unlimitedAnnualFee = unlimited.feePesewas * (unlimited.billingPeriod === 'annual' ? 1 : 12);

  const gap = unlimitedAnnualFee - meteredAnnualFee;
  if (gap <= 0) return 0;
  return Math.ceil(gap / perDownload);
}
```

The price of one more download belongs on the download button. An organisation should never discover the price of a report on next month's invoice.

Note that standard's annual fee (GHS 21,600) already sits close to enterprise's (GHS 24,000), so standard breaks even at far fewer downloads than basic. That is intentional and the console shows the arithmetic rather than a sales claim.

10.2 Reporter commission

What a reporter earns when an institution licenses their report.

Base rate per category, in pesewas. Deliberately uneven — a fire or a road accident has a short window in which it is worth anything to a responder, and paying the same for it as for a pothole would tell reporters their urgency is worthless.

CATEGORY	BASE	CATEGORY	BASE
corruption	3,000	utility	1,500
election	3,000	water	1,500
galamsey	3,000	education	1,500
fire	2,500	wildlife	1,500
accident	2,500	sanitation	1,400
environment	2,200	transport	1,300
flood	2,000	road	1,200
crime	2,000	infrastructure	1,200
disorder	2,000	other	1,000
chieftaincy	2,000		
weather	1,800		
health	1,800		
protest	1,800		
land	1,800		

Governance categories pay most: hardest to capture, most consequential once captured, and the footage most likely to cost the reporter something to obtain. A lawful demonstration (`protest`) sits below `disorder` because it is newsworthy but rarely time-critical to a responder.

Multipliers

MULTIPLIER	VALUE	APPLIES WHEN
Video	×1.5	<code>mediaKind === 'video'</code>
Audio	×1.25	<code>mediaKind === 'audio'</code>
Directed	×1.25	<code>destination === 'directed'</code> — exclusivity is worth more
Low confidence	×0.7	<code>locationConfidence === 'low'</code>
Additional licensees	+0.5 share each	Second and subsequent buyers

Audio sits between a photograph and a video on purpose. Speaking an account takes more of a reporter than pointing a lens, so it pays above a still; it shows less than footage, so it pays below one. Pricing it like a photo would tell people the safest way to report is the least valuable, which is the opposite of what this platform should encourage.

Platform share: `PLATFORM_FEE_RATE = 0.3`

```
export function estimateCommission(input: CommissionInput): CommissionBreakdown {
  // The public feed pays nothing. Visibility is the reward, and pretending
  // otherwise would set an expectation the model cannot meet.
  if (input.destination === 'public') {
    return { grossPesewas: 0, platformFeePesewas: 0, reporterPesewas: 0 };
  }

  let gross = BASE_PESEWAS[input.category];
  if (input.mediaKind === 'video') gross *= 1.5;
  if (input.mediaKind === 'audio') gross *= 1.25;
  if (input.destination === 'directed') gross *= 1.25;
  if (input.locationConfidence === 'low') gross *= 0.7;

  // The second buyer of the same footage values it less than the first, but
  // the reporter should still gain.
  const licensees = Math.max(1, input.licensedBy ?? 1);
  gross *= 1 + (licensees - 1) * 0.5;

  // Round ONCE, at the end. Rounding each multiplier compounds the error.
  const grossPesewas = Math.round(gross);
  const platformFeePesewas = Math.round(grossPesewas * 0.3);

  return {
    grossPesewas,
    platformFeePesewas,
    // Subtraction, not a second rounding, so the three always reconcile.
    reporterPesewas: grossPesewas - platformFeePesewas,
  };
}
```

The same function produces the estimate shown to the reporter before they submit and the amount recorded in the ledger afterwards. They must never disagree. If you reimplement this server-side, verify it against §15.

10.3 The ledger and payouts

```
export type CommissionStatus =
  | 'pending' // submitted; nobody has licensed it yet
  | 'earned'  // a business licensed it – the amount is fixed at this point
  | 'paid'    // included in a payout batch
  | 'void';   // report rejected or withdrawn; nothing owed

export interface EarningsSummary {
  pendingPesewas: number;
  paidPesewas: number;
  lifetimePesewas: number;
  reportsLicensed: number;
  payoutThresholdPesewas: number; // payouts run once a reporter clears this floor
  nextPayoutIso: string | null;
}
```

The amount is **fixed at the moment of licensing** and MUST NOT be recomputed later. Rates change; a reporter's settled earnings do not.

Mobile money is how Ghana pays. Payout destinations will be MTN MoMo, Telecel Cash and AirtelTigo Money. The payout provider integration is not yet specified — see §16.

11. Response and SLA

Until now a subscriber could license footage and nothing more, and the platform had no idea whether anyone acted on it. That gap matters in both directions: a reporter who never learns anything happened stops filing, and an agency that cannot show it responded has no evidence of its own work.

The response log is append-only. "We closed this with no action" is a decision someone made and should have to stand behind, not something that can be quietly rewritten later.

```
export type ResponseAction =
  | 'acknowledged' // seen by a human; nothing done yet
  | 'more_info_requested' // cannot act without something more from the reporter
  | 'inspecting' // someone has gone to look
  | 'referred' // passed to a different body
  | 'resolved' // dealt with
  | 'closed_no_action'; // considered and deliberately not acted on
```

ACTION	TERMINAL	NOTIFIES REPORTER
acknowledged	no	✓
more_info_requested	no	✓
inspecting	no	✓
referred	yes	✓
resolved	yes	✓
closed_no_action	yes	✓

Every action notifies the reporter, including the ones nobody enjoys sending.

Acknowledgement targets

```
export const ACK_TARGET_HOURS: Record<Severity, number> = {
  emergency: 1,
  urgent: 4,
  concern: 24,
  observation: 72,
};

export type SlaStatus = 'met' | 'due' | 'at_risk' | 'breached';
```

```
export function slaState(severity, submittedAtIso, acknowledgedAtIso, nowIso): SlaState {
  const targetHours = ACK_TARGET_HOURS[severity];
  const submitted = Date.parse(submittedAtIso);
  // Once acknowledged the clock STOPS. Recomputing against the current time
  // would make met cases silently drift into breached ones.
  const reference = Date.parse(acknowledgedAtIso ?? nowIso);

  const elapsed = Math.max(0, (reference - submitted) / 3_600_000);
  const hoursRemaining = targetHours - elapsed;

  if (acknowledgedAtIso) {
    return { status: elapsed <= targetHours ? 'met' : 'breached', targetHours, hoursRemaining };
  }
  if (hoursRemaining <= 0) return { status: 'breached', targetHours, hoursRemaining };
  // The last quarter of the window is where a nudge still changes the outcome.
  if (hoursRemaining <= targetHours * 0.25) {
    return { status: 'at_risk', targetHours, hoursRemaining };
  }
  return { status: 'due', targetHours, hoursRemaining };
}

/** Only UNACKNOWLEDGED breaches escalate – a case acknowledged late is already
 * with a person, and escalating would tell a supervisor to chase work in hand. */
export function needsEscalation(sla: SlaState, acknowledged: boolean): boolean {
  return sla.status === 'breached' && !acknowledged;
}
```

12. Surveys

Institutions can pay reporters to answer structured questions from a target area.

```
export type SurveyQuestionKind = 'single_choice' | 'multi_choice' | 'scale' | 'text' | 'photo';

export interface Survey {
  id: string;
  businessId: string;
  businessName: string;
  title: string;
  description: string;
  questions: SurveyQuestion[];
  rewardPesewas: number;
  targetArea: { latitude: number; longitude: number; radiusM: number } | null;
  responsesTarget: number;
  responsesReceived: number;
  closesAtIso: string;
  status: 'draft' | 'live' | 'closed';
}
```

Constants: `MIN_REWARD_PESEWAS = 100` (GHS 1 — below this a survey is not worth a reporter's attention).

`MAX_QUESTIONS = 12` (answering more in one sitting produces careless data).

Validation returns **everything** wrong rather than the first problem found:

```
export type SurveyIssue =
  | 'no_title' | 'no_questions' | 'question_missing_prompt'
  | 'choice_needs_options' | 'reward_too_low' | 'no_target' | 'closes_in_past';
```

Cost, with the same platform share as commission:

```
export function estimateSurveyCost(rewardPesewas: number, responsesTarget: number): SurveyCost {
  const rewardsPesewas = Math.max(0, Math.round(rewardPesewas * responsesTarget));
  const platformFeePesewas = Math.round(rewardsPesewas * PLATFORM_FEE_RATE);
  return { rewardsPesewas, platformFeePesewas, totalPesewas: rewardsPesewas + platformFeePesewas };
}
```

A business's concurrent live surveys MUST NOT exceed its plan's `concurrentSurveys`.

13. Endpoint index

Endpoints marked **new** do not exist in v1 of this contract and cover everything built since.

Capture and identity

METHOD	PATH	AUTH
POST	/devices	none
POST	/auth/register	none
POST	/auth/login	none
POST	/auth/refresh	refresh token
POST	/auth/google	none — new , currently non-functional in the client
POST	/incidents	device or user
PUT	/uploads/{uploadId}/chunks/{index}	same as init
POST	/uploads/{uploadId}/complete	same as init
GET	/uploads/{uploadId}	same as init

Reading

METHOD	PATH	AUTH
GET	/incidents	device or user
GET	/incidents/map	device or user
GET	/incidents/{id}	device or user
GET	/incidents/by-reference/{reportId}	none — new , public verification page
GET	/me/incidents	user
PATCH	/incidents/{id}	user, author only
DELETE	/incidents/{id}	user, author only
GET	/incidents/{id}/comments	device or user — new
POST	/incidents/{id}/comments	user — new
POST	/incidents/{id}/reactions	user — new
DELETE	/incidents/{id}/reactions	user — new

Editorial — all new

METHOD	PATH	CAPABILITY
GET	/editorial/queue	editor
GET	/editorial/{incidentId}	editor
POST	/editorial/{incidentId}/transition	editor — enforces §4.3
POST	/editorial/{incidentId}/corroboration	editor
GET	/editorial/decided	editor

Institution

METHOD	PATH	CAPABILITY
GET	/org/dashboard	view_reports
GET	/org/inbox	view_reports
POST	/org/incidents/{id}/license	view_reports — enforces \$4.5
POST	/org/incidents/{id}/publish	manage_org — enforces \$4.5
POST	/org/incidents/{id}/response	triage_reports — new , \$11
GET/POST/DELETE	/org/queries/{id}	manage_queries
POST	/org/exports	export_data
GET/POST/PATCH/DELETE	/org/members/{id}	manage_members
GET/POST/PATCH/DELETE	/org/branches/{id}	manage_org — new
GET/POST/PATCH/DELETE	/org/employees/{id}	manage_members — new
GET	/org/membership-requests	manage_members — new
POST	/org/membership-requests/{id}/decide	manage_members — new
POST	/org/assignments	dispatch_units — new , \$7
GET/POST	/org/internal-submissions	view_reports — new , \$8.6
GET/POST/DELETE	/org/invites/{id}	manage_members — new
GET/POST/DELETE	/org/affiliations/{id}	manage_org — new
GET/POST	/org/surveys	manage_queries
GET	/org/onboarding	manage_org — new
PUT	/org/onboarding/steps/{stepId}	manage_org — new
POST	/org/onboarding/steps/{stepId}/submit	manage_org — new
POST	/org/onboarding/documents	manage_org — new

Platform operator — all new

METHOD	PATH
GET	/platform/routing
POST	/platform/routing/{incidentId}/recipients
GET	/platform/applications
POST	/platform/applications/{id}/steps/{stepId}/decide
POST	/platform/applications/{id}/screening
POST	/platform/applications/{id}/approve
GET	/platform/businesses
GET	/platform/payouts
POST	/platform/payouts/batches

Reporter earnings

METHOD	PATH	AUTH
GET	/me/earnings	user
GET	/me/commissions	user
GET	/surveys	user
POST	/surveys/{id}/responses	user

14. Invariants — the must-never list

Every one of these is a rule the clients already follow and the server MUST enforce independently. They are collected here because they are the failures that would matter, and because an implementation that gets everything else right and one of these wrong is worse than useless.

1. **Never let assurance imply verification.** A Class A file is not a verified event. The two fields are separate, stored separately, and no code path may set one from the other.
2. **Never use the word "verified" against a state whose `mayUseWordVerified` is false.** Return `permittedRepresentation` and have clients render it verbatim.
3. **Never allow a transition outside the table in §4.3**, and never allow anything out of `rejected`.
4. **Never publish a report whose handling includes `redact_before_publication`** until redaction is done and recorded — regardless of verification state.
5. **Never publish a Class C report that is not independently corroborated**, however far editorial has got.
6. **Never return `PreciseLocation` on a public endpoint.** Author-only, always.

7. **Never return suppressed fields.** `showLocation: false` means coordinates and label are null in the response body, not merely hidden by the client.
8. **Never trust a client-supplied assurance class, handling requirement, commission amount, or capture timestamp.** Compute all of them server-side.
9. **Never store money as a float.** Integer pesewas everywhere, rounded once, reconciled by subtraction.
10. **Never recompute a settled commission.** The amount is fixed at licensing.
11. **Never let an admin modify or remove an owner,** and never allow the last owner to be removed.
12. **Never let an organisation approve its own onboarding.** Approval is always an act by a platform administrator, and only after screening is run and clear.
13. **Never treat `severity: 'emergency'` as a dispatch to an emergency service.** Dawuro is not one. Ghana's emergency number is 112.
14. **Never make an internal submission anonymous.** Attribution is the point.
15. **Never return a retryable error code for a permanent failure,** or the reverse. The mobile outbox switches on it.
16. **Never route a `public` submission to institutions,** and never pay commission on one.
17. **Never infer an employee's location from their branch address.**
18. **Never let the SLA clock restart.** Once acknowledged, the answer is fixed.
19. **Never delete from the response log.** Append-only.
20. **Never accept an organisation context from anything but a verified membership** on the authenticated user.

15. Test vectors

Verify your implementation against these before wiring anything to a client.

Report ID

SEED	EXPECTED
<code>inc_01JBX7Q2K9</code>	<code>DW-WQD-JW4</code>
<code>inc_01JBX7R4M2</code>	<code>DW-YNG-6JR</code>
<code>inc_01JBX7S6P7</code>	<code>DW-VPR-WCH</code>
<code>inc_01JBX7T8Q1</code>	<code>DW-DC9-VKM</code>

Lookup MUST be case-insensitive: `dw-wqd-jw4` resolves the same report.

Assurance classification

CAPTUREDINAPP	INSTITUTIONAL	ALL 5 CHECKS	→ CLASS
false	true	true	C — an import can never be rescued
false	false	false	C
true	true	false	D
true	false	true	A
true	false	false	B

Commission

INPUT	GROSS	FEE	REPORTER
<code>flood</code> , marketplace, photo, high	2,000	600	1,400
<code>flood</code> , marketplace, video, high	3,000	900	2,100
<code>flood</code> , marketplace, audio, high	2,500	750	1,750
<code>flood</code> , directed, video, high	3,750	1,125	2,625
<code>flood</code> , marketplace, video, low	2,100	630	1,470
<code>galamsey</code> , directed, video, high	5,625	1,688	3,937
<code>flood</code> , both , video, high	3,000	900	2,100
anything, public , any, any	0	0	0
<code>flood</code> , marketplace, video, high, 2 licensees	4,500	1,350	3,150

The three figures MUST always satisfy `gross - fee = reporter` exactly.

Billing

ASSERTION	VALUE
<code>annualCost(basic, 0)</code>	540,000
<code>annualCost(standard, 0)</code>	2,160,000
<code>annualCost(enterprise, 0)</code>	2,400,000
<code>annualCost(enterprise, 10_000)</code>	2,400,000 — unlimited
<code>breakEvenDownloads(basic, enterprise)</code>	930
<code>breakEvenDownloads(standard, enterprise)</code>	200
<code>breakEvenDownloads(enterprise, enterprise)</code>	<code>null</code>

Standard breaks even sooner than basic. That is correct, not a bug: standard's annual fee already sits near enterprise's.

SLA

SEVERITY	SUBMITTED	ACKNOWLEDGED	NOW	→ STATUS
emergency	T+0	—	T+0.5h	due — half the window still left
emergency	T+0	—	T+0.8h	at_risk — ≤25% of a 1h window left
emergency	T+0	—	T+2h	breached
emergency	T+0	T+3h	T+99h	breached — the clock stopped at acknowledgement
observation	T+0	T+70h	T+99h	met
concern	T+0	—	T+1h	due

Handling requirements

CONSENT	→ HANDLING
containsMinors	redact_before_publication, editorial_review_required
distressing	viewer_warning, editorial_review_required
showsPrivateProperty only	restrict_location, redact_before_publication
showsPrivateProperty + publicPlace	restrict_location
showsPrivateProperty + subjectsConsented	restrict_location
all false	[]

16. Open decisions

These need a decision from the product side before or during implementation. They are listed here rather than guessed at.

1. **Device clock tolerance.** What gap between `capturedAtIso` and server-received time sets `timeCheckPassed` false? Suggested 5 minutes plus observed upload delay.
2. **GPS accuracy limits per category.** `GPS_ACCURACY_REJECTED` implies a threshold. Does a `fire` demand a tighter fix than a `sanitation` report?
3. **Media retention.** How long is media held? The mobile client keeps a metadata row after deleting its local copy, so a reporter's history will eventually point at collected media. What should it show then?
4. **Payout provider.** MTN MoMo, Telecel Cash, AirtelTigo — which, and through which aggregator? What is `payoutThresholdPesewas` and how often do batches run?
5. **Key management.** Where does the server-side integrity signing key live, how is it rotated, and how are historical signatures verified after a rotation?
6. **Audit ledger.** The proposal calls for a tamper-evident record of every editorial decision. Append-only table, hash chain, or an external service?

7. **Right of reply and takedown.** Ghana's Data Protection Act (Act 843) applies. What is the process when someone filmed requests removal, and what happens to a report already licensed and published by a third party?
8. **Reporter deanonymisation risk.** A timestamp plus coordinates plus a distinctive scene can identify an "anonymous" reporter. Is there a policy on suppressing detail for high-risk categories such as `galamsey` and `corruption`?
9. **Rate limits.** The figures in §2.8 are starting points, not measurements.
10. `reportId` **collisions.** Confirm the persistence-with-uniqueness approach in §3.7.

Appendix — reference implementation

Every algorithm in this document is implemented and unit-tested in TypeScript in `packages/core` of the console repository, with no React, DOM or Node dependencies. If a rule here is ambiguous, that code is the tiebreaker — and if the two disagree, **this document is what the backend builds against** and both should be corrected together.

Consult it for: `assignment.ts` (\$7 scoring in full), `editorial.ts` (\$4.4 corroboration), `clusters.ts` (map clustering for `GET /incidents/map`), `surveyLogic.ts` (\$12 validation).